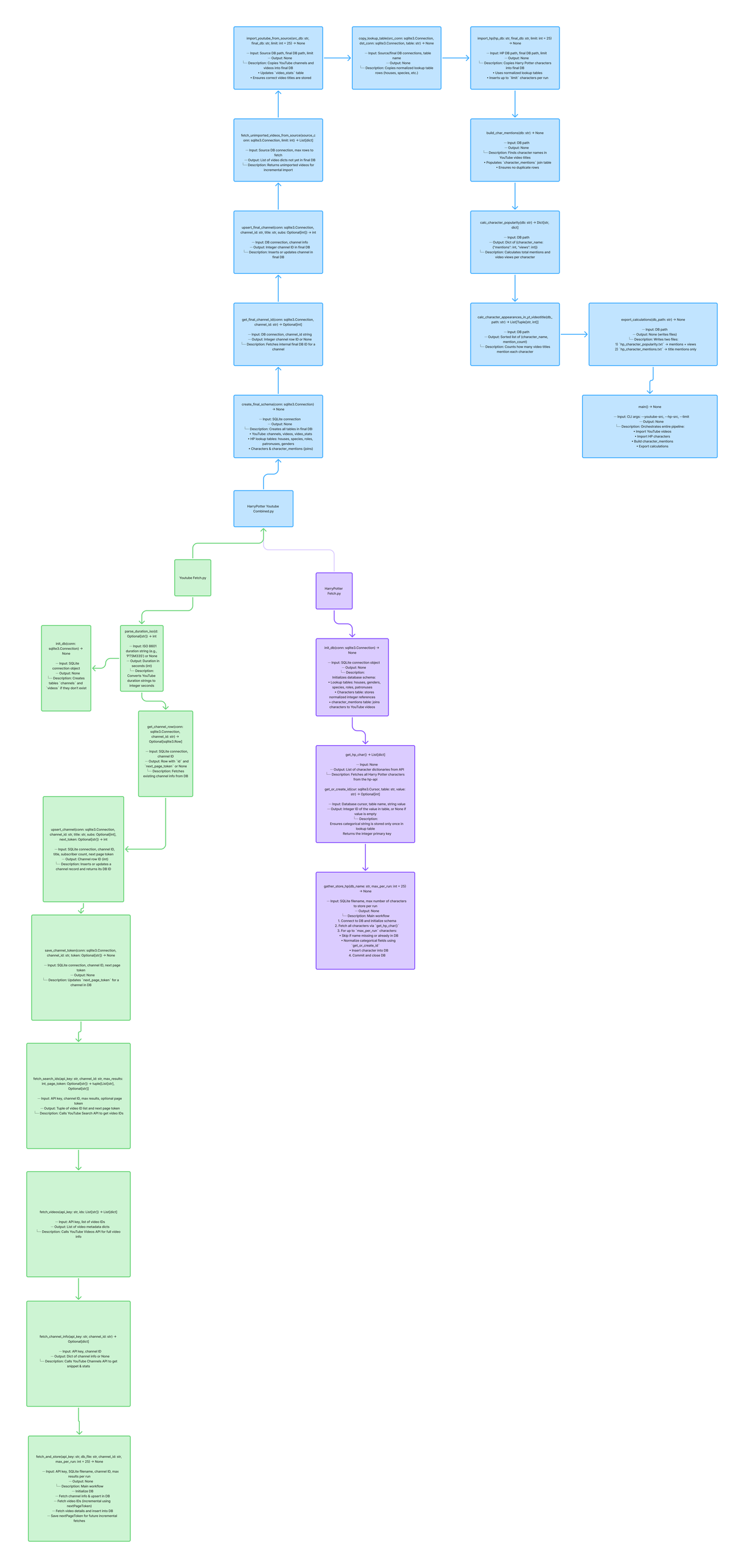
import_youtube_from_source(src_db: str, final_db: str, limit: int = 25) → None
— Input: Source DB path, final DB path, limit
— Output: None
└ Description: Copies YouTube channels and videos into final DB
• Updates `video_stats` table
• Ensures correct video titles are stored
copy_lookup_table(src_conn: sqlite3.Connection, dst_conn: sqlite3.Connection, table: str) → None
— Input: Source/final DB connections, table name
— Output: None
└ Description: Copies normalized lookup table rows (houses, species, etc.)
import_hp(hp_db: str, final_db: str, limit: int = 25) → None
— Input: HP DB path, final DB path, limit
— Output: None
└ Description: Copies Harry Potter characters into final DB
• Uses normalized lookup tables
• Inserts up to `limit` characters per run
fetch_uninported_videos_from_source(source_conn: sqlite3.Connection, limit: int) → List[dict]
— Input: Source DB connection, max rows to fetch
— Output: List of video dicts not yet in final DB
└ Description: Returns unimported videos for incremental import
build_char_mentions(db: str) → None
— Input: DB path
— Output: None
└ Description: Finds character names in YouTube video titles
• Populates `character_mentions` join table
• Ensures no duplicate rows
upsert_final_channel(conn: sqlite3.Connection, channel_id: str, title: str, subs: Optional[int]) → int
— Input: DB connection, channel info
— Output: Integer channel ID in final DB
└ Description: Inserts or updates channel in final DB
calc_character_popularity(db: str) → Dict[str, dict]
— Input: DB path
— Output: Dict of {character_name: {"mentions": int, "views": int}}
└ Description: Calculates total mentions and video views per character
get_final_channel_id(conn: sqlite3.Connection, channel_id: str) → Optional[int]
— Input: DB connection, channel_id string
— Output: Integer channel row ID or None
└ Description: Fetches internal final DB ID for a channel
calc_character_appearances_in_yt_videotitle(db_path: str) → List[Tuple[str, int]]
— Input: DB path
— Output: Sorted list of (character_name, mention_count)
└ Description: Counts how many video titles mention each character
export_calculations(db_path: str) → None
— Input: DB path
— Output: None (writes files)
└ Description: Writes two files:
1) `hp_character_popularity.txt` → mentions + views
2) `hp_character_mentions.txt` → title mentions only
create_final_schema(conn: sqlite3.Connection) → None
— Input: SQLite connection
— Output: None
└ Description: Creates all tables in final DB:
• YouTube: channels, videos, video_stats
• HP lookup tables: houses, species, roles, patronuses, genders
• Characters & character_mentions (joins)
main() → None
— Input: CLI args: --youtube-src, --hp-src, --limit
— Output: None
└ Description: Orchestrates entire pipeline:
• Import YouTube videos
• Import HP characters
• Build character_mentions
• Export calculations
HarryPotter Youtube Combined.py
Youtube Fetch.py
HarryPotter Fetch.py
init_db(conn: sqlite3.Connection) → None
— Input: SQLite connection object
— Output: None
└ Description: Creates tables `channels` and `videos` if they don't exist
parse_duration_iso(d: Optional[str]) → int
— Input: ISO 8601 duration string (e.g., 'PT5M33S') or None
— Output: Duration in seconds (int)
└ Description: Converts YouTube duration strings to integer seconds
get_channel_row(conn: sqlite3.Connection, channel_id: str) → Optional[sqlite3.Row]
— Input: SQLite connection, channel ID
— Output: Row with `id` and `next_page_token` or None
└ Description: Fetches existing channel info from DB
upsert_channel(conn: sqlite3.Connection, channel_id: str, title: str, subs: Optional[int], next_token: Optional[str]) → int
— Input: SQLite connection, channel ID, title, subscriber count, next page token
— Output: Channel row ID (int)
└ Description: Inserts or updates a channel record and returns its DB ID
save_channel_token(conn: sqlite3.Connection, channel_id: str, token: Optional[str]) → None
— Input: SQLite connection, channel ID, next page token
— Output: None
└ Description: Updates `next_page_token` for a channel in DB
fetch_search_ids(api_key: str, channel_id: str, max_results: int, page_token: Optional[str]) → tuple[List[str], Optional[str]]
— Input: API key, channel ID, max results, optional page token
— Output: Tuple of video ID list and next page token
└ Description: Calls YouTube Search API to get video IDs
fetch_videos(api_key: str, ids: List[str]) → List[dict]
— Input: API key, list of video IDs
— Output: List of video metadata dicts
└ Description: Calls YouTube Videos API for full video info
fetch_channel_info(api_key: str, channel_id: str) → Optional[dict]
— Input: API key, channel ID
— Output: Dict of channel info or None
└ Description: Calls YouTube Channels API to get snippet & stats
fetch_and_store(api_key: str, db_file: str, channel_id: str, max_per_run: int = 25) → None
— Input: API key, SQLite filename, channel ID, max results per run
— Output: None
└ Description: Main workflow
— Initialize DB
— Fetch channel info & upsert in DB
— Fetch video IDs (incremental using nextPageToken)
— Fetch video details and insert into DB
— Save nextPageToken for future incremental fetches
init_db(conn: sqlite3.Connection) → None
— Input: SQLite connection object
— Output: None
└ Description:
Initializes database schema:
• Lookup tables: houses, genders, species, roles, patronuses
• Characters table: stores normalized integer references
• character_mentions table: joins characters to YouTube videos
get_hp_char() → List[dict]
— Input: None
— Output: List of character dictionaries from API
└ Description: Fetches all Harry Potter characters from the hp-api
get_or_create_id(cur: sqlite3.Cursor, table: str, value: str) → Optional[int]
— Input: Database cursor, table name, string value
— Output: Integer ID of the value in table, or None if value is empty
— Description:
Ensures categorical string is stored only once in lookup table
Returns the integer primary key
gather_store_hp(db_name: str, max_per_run: int = 25) → None
— Input: SQLite filename, max number of characters to store per run
— Output: None
└ Description: Main workflow
1. Connect to DB and initialize schema
2. Fetch all characters via `get_hp_char()`
3. For up to `max_per_run` characters:
• Skip if name missing or already in DB
• Normalize categorical fields using `get_or_create_id`
• Insert character into DB
4. Commit and close DB